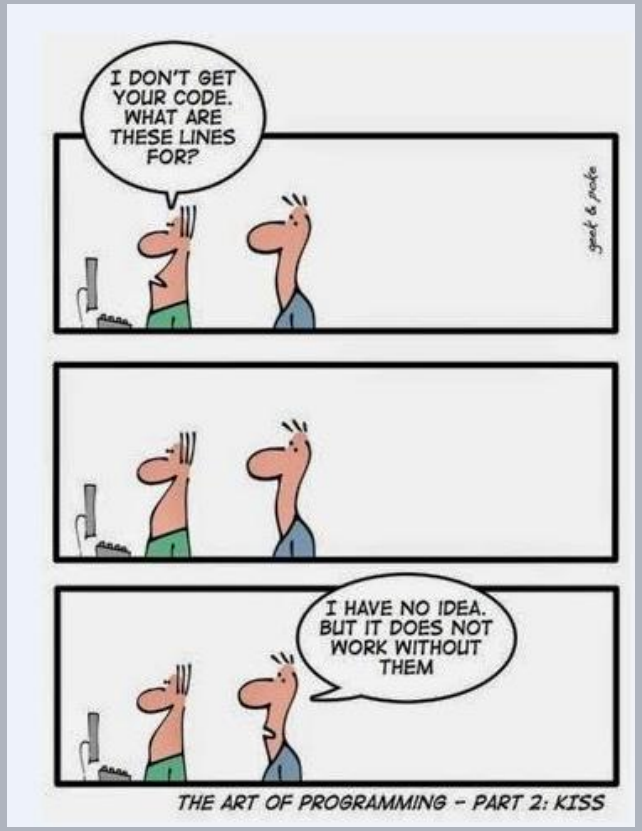
Advanced Software Design Techniques

Interface Design

Prof. Dr.-Ing. Peter Fromm



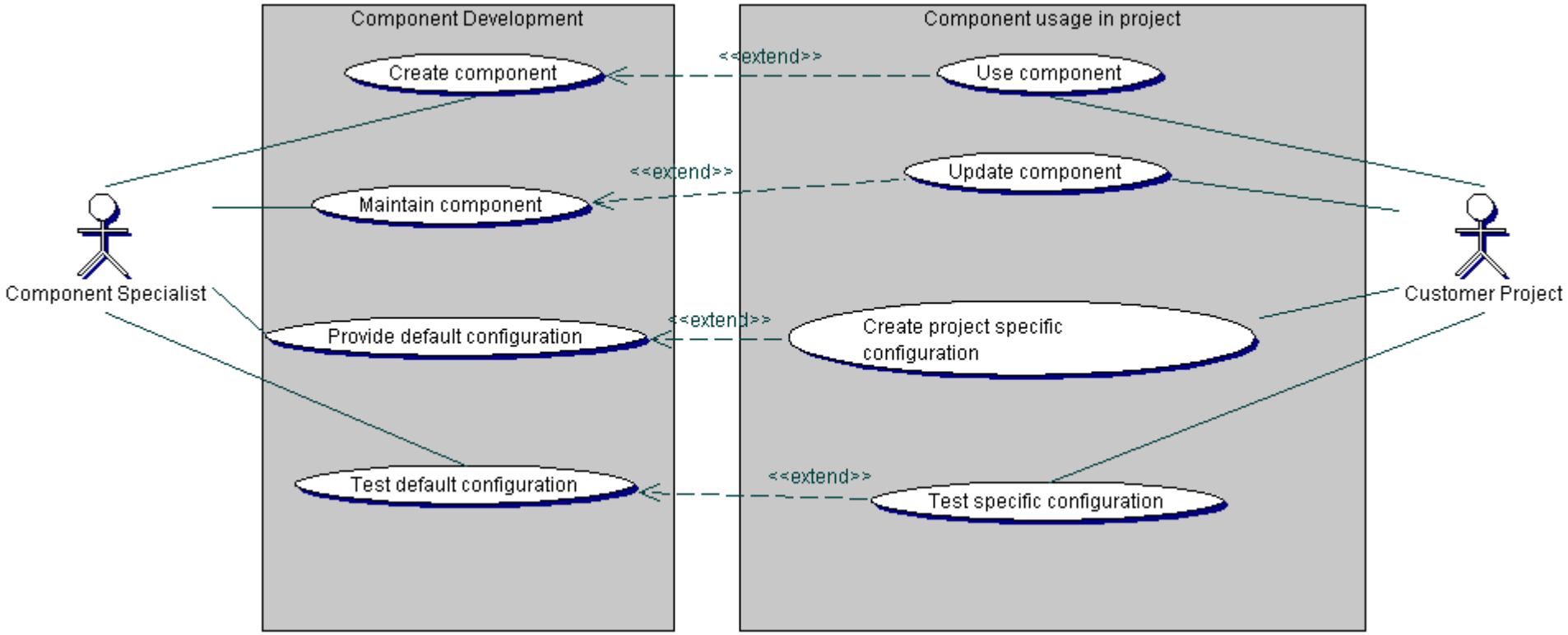
Content

- Interface Design
  - Component Level
  - Class Level
  - Function Level
- Function Pointers and Functors
- Stubbing
- Component Configuration
- Release Processes

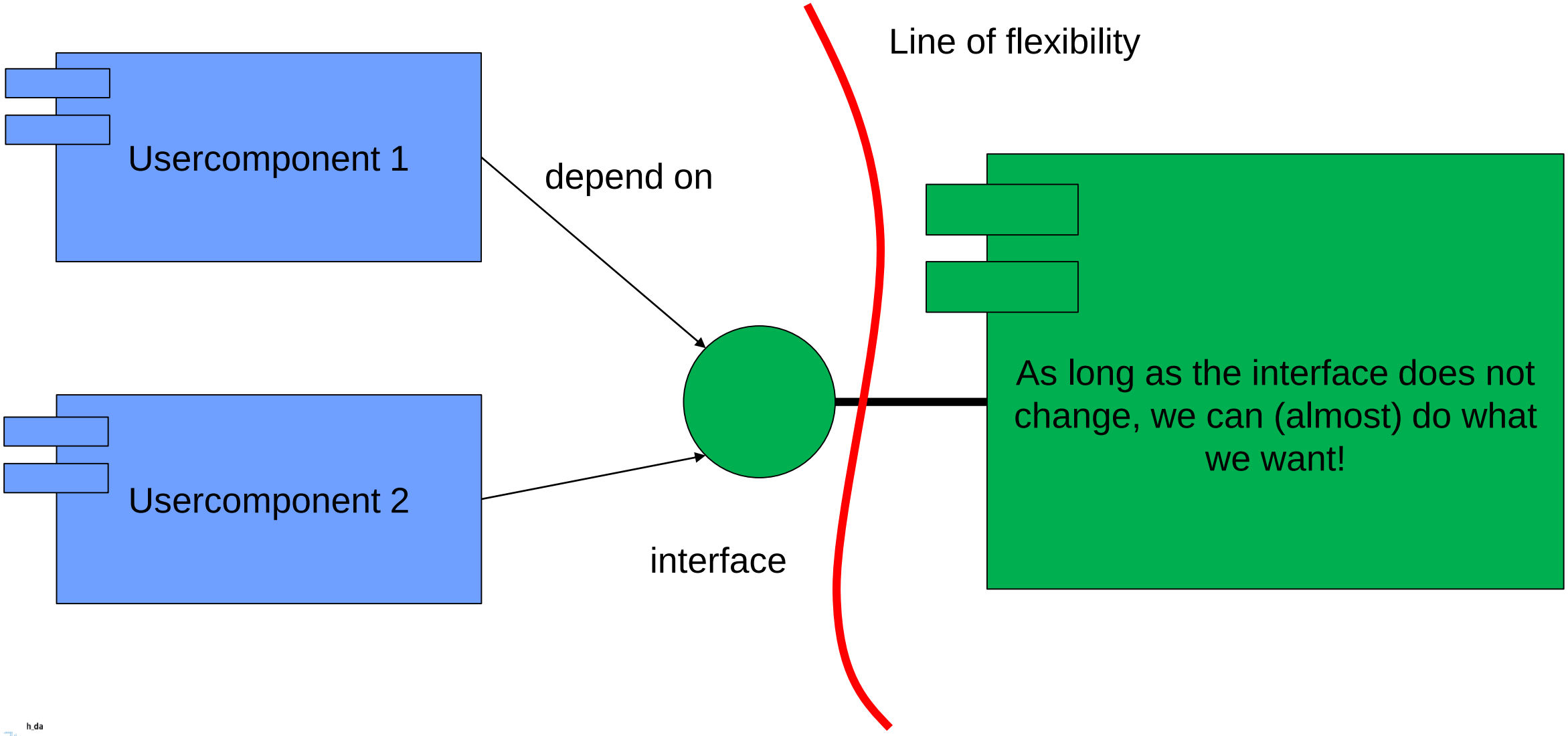
Component based development

In large companies, teams often work following a component based approach

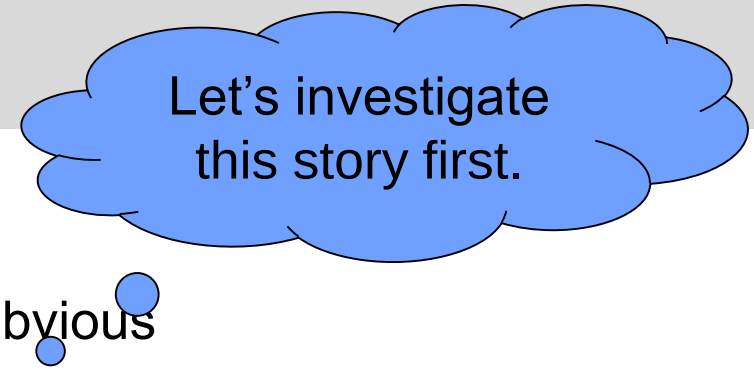
- Usage shall be easy AND flexible
- Update shall be easy
- Additional tests shall be minimized



Importance of Interfaces



Some ideas about components and component reuse



When are we willing to (re)use components?

- The component has to provide the required functionality - that's obvious
- The component usage shall be easy – that's more challenging

- The classes / sources which form the component must go through a release process
- The class releases must form a working component release. The classes have to be compatible!
- This implies a strong cohesion inside the component. It has to make sense, that these classes are released together.

- Finally, the developer will decide if he/she goes for a new release or not.
- This implies also a certain compatibility between the releases.

What makes an interface a good interface?

```
class CUart_v1
{
public:
enum port_t{ASCLIN1, ASCLIN2, ASCLIN3,
   ASCLIN5, ASCLIN5, NONE};

enum parity_t{ODD, EVEN, NONE};

CUart_v1(port_t port = NONE,
         uint16_t baudrate = 115200,
         uint8_t bits = 8,
         parity_t parity = NONE,
         uint8_t stopbits = 1);
};
```

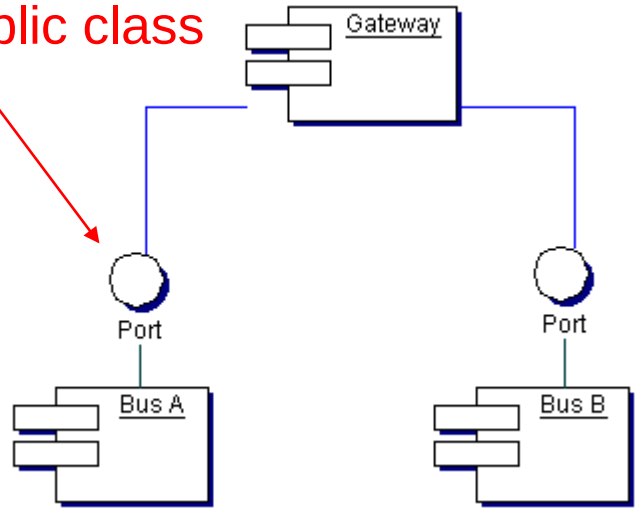
```
class CUart_v2
{
public:

   CUart_v2(uint16_t *const configReg,
            uint16_t *const baudReg1,
            uint16_t *const baudReg2,
            uint16_t configRegValue,
            int16_t baudReg1Value,
            uint16_t baudReg2Value);
};
```

A good interface…

- Is **unique** in the system – there is no confusion, on whether it should be used or not
- Is **minimal** – only the required functions shall, be provided, making it easy to be understood
- Is **descriptive** – good names and types provide a clear usage guideline
- Is **abstract, hiding implementation** details – enabling the developer to change the implementation without affecting the interface.
- Is **robust and acting standalone** – there should be no hidden sideffects / requirements when using it (i.e. function A has to be called before function B, otherwise B might crash), making it safe to be used
- Is **providing clear status information** – error conditions must be clearly returned

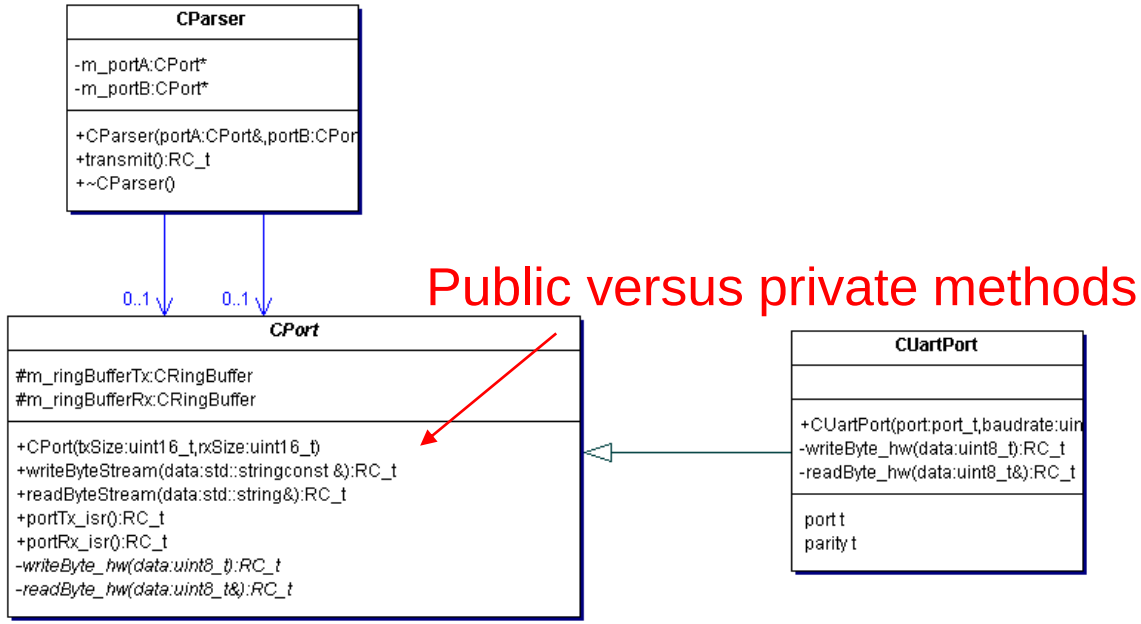
Different Levels of Interfaces

Public class




```
/**
 * \brief Constructor - Will initialize the peripheral port and setup the RX and TX buffers
 *
 *  Constructor - Will initialize the peripheral port and setup the RX and TX buffers
 *  Please check the hardware limitations to ensure proper configuration parameters
 *
 *  \param port_t port = NONE           : IN    The hardware port to be used
 *  \param uint16_t baudrate = 115200   : IN    The baudrate of the device. Check the hardware manual for valid values.
 *  \param uint8_t bits = 8             : IN    The number of data bits, typically 7 or 8
 *  \param parity_t parity = NONE       : IN    Parity, ODD, EVEN or NONE
 *  \param uint8_t stopbits = 1         : IN    Number of Stopbits, 1 or 2
 *  \param uint16_t bufferSizeRx = UART_DEFAULTBUFFERSIZE : IN  Size of the Receive Buffer
 *  \param uint16_t bufferSizeTx = UART_DEFAULTBUFFERSIZE : IN Size of the Transmit Buffer
 */
CUartPort(port_t port = NONE,
          uint32_t baudrate = 115200,
          uint8_t bits = 8,
          parity_t parity = NOPARITY,
          uint8_t stopbits = 1,
          uint16_t bufferSizeRx = UART_DEFAULTBUFFERSIZE,
          uint16_t bufferSizeTx = UART_DEFAULTBUFFERSIZE
          );
```

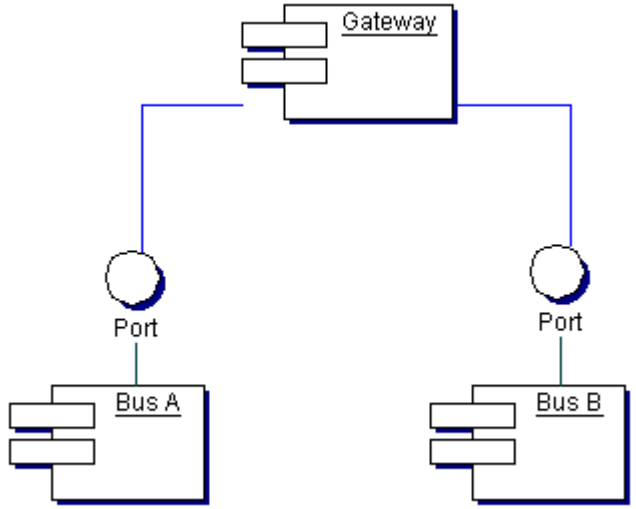
API Design



Public versus private methods

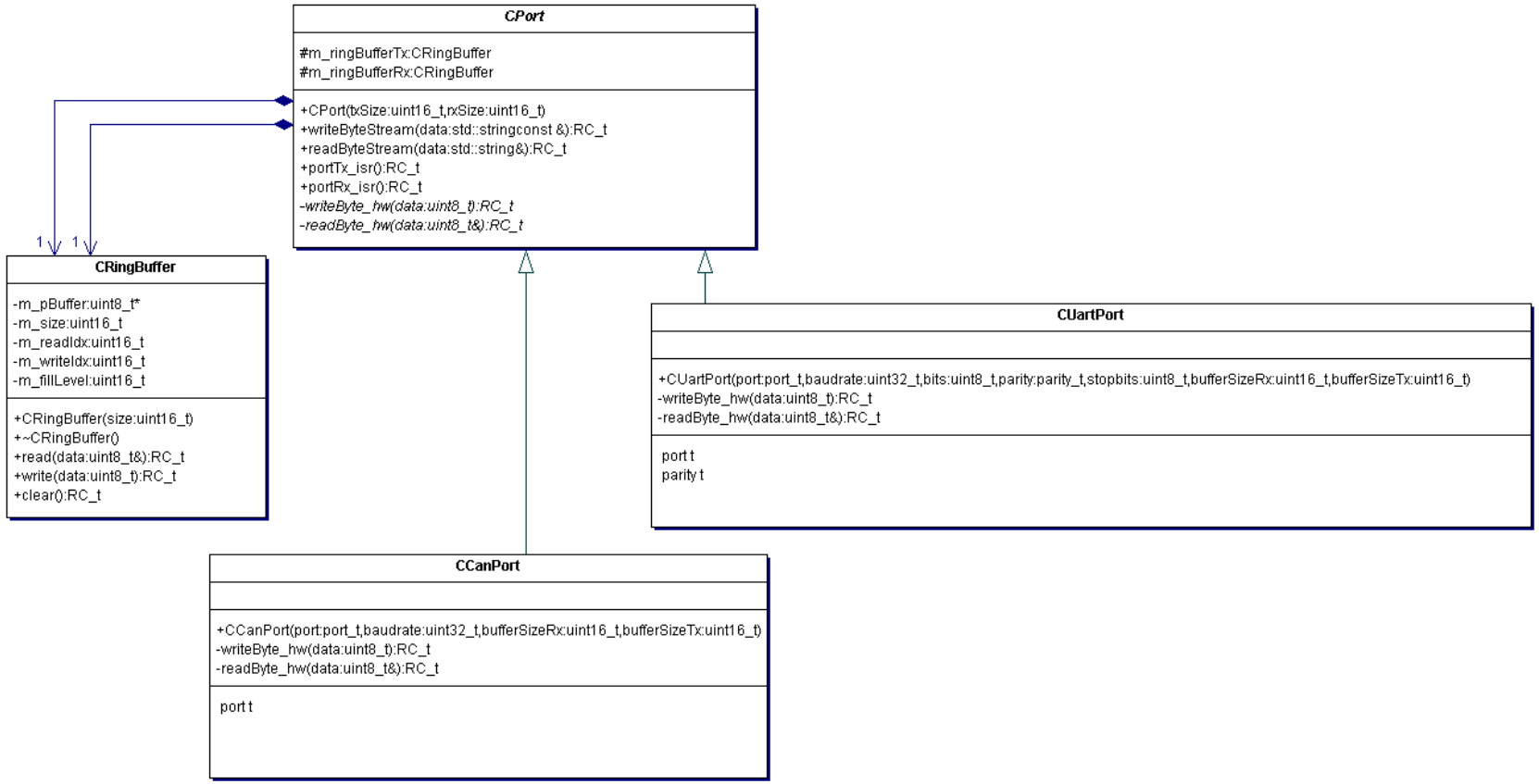
Design of a gateway system
Component and Class Level

- The task of the gateway component is to read data from one bus, translate the protocol and send it out through another bus.
- The system should be flexible and support different hardware busses.
- Data transfer will be performed as bytestream (aka string).



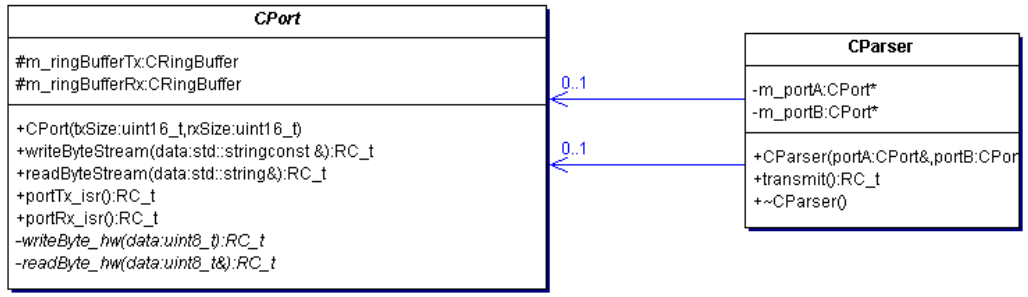
Class Design – Generic Port

- The system shall communicate through a generic port class, which will provide the interface
- A concrete class (in our example `CUartPort` and `CCanPort`) will provide the hardware related interfaces (but only those)



Port Component Usage

- Except for creating the physical devices, the user may ignore any hardware related parts.
- The parser itself only works with the `CPort` API's.




```
RC_t CParser::transmit()
{

    if (0 == m_portA || 0 == m_portB)
    {
        return RC_ERROR;
    }

    RC_t result = RC_ERROR;
    //Transmit from A to B
    string data;

    //Read the data
    result = m_portA->readByteStream(data);

    //Do something with the data

    //Write the data
    result = m_portB->writeByteStream(data);

    return result;
}
```

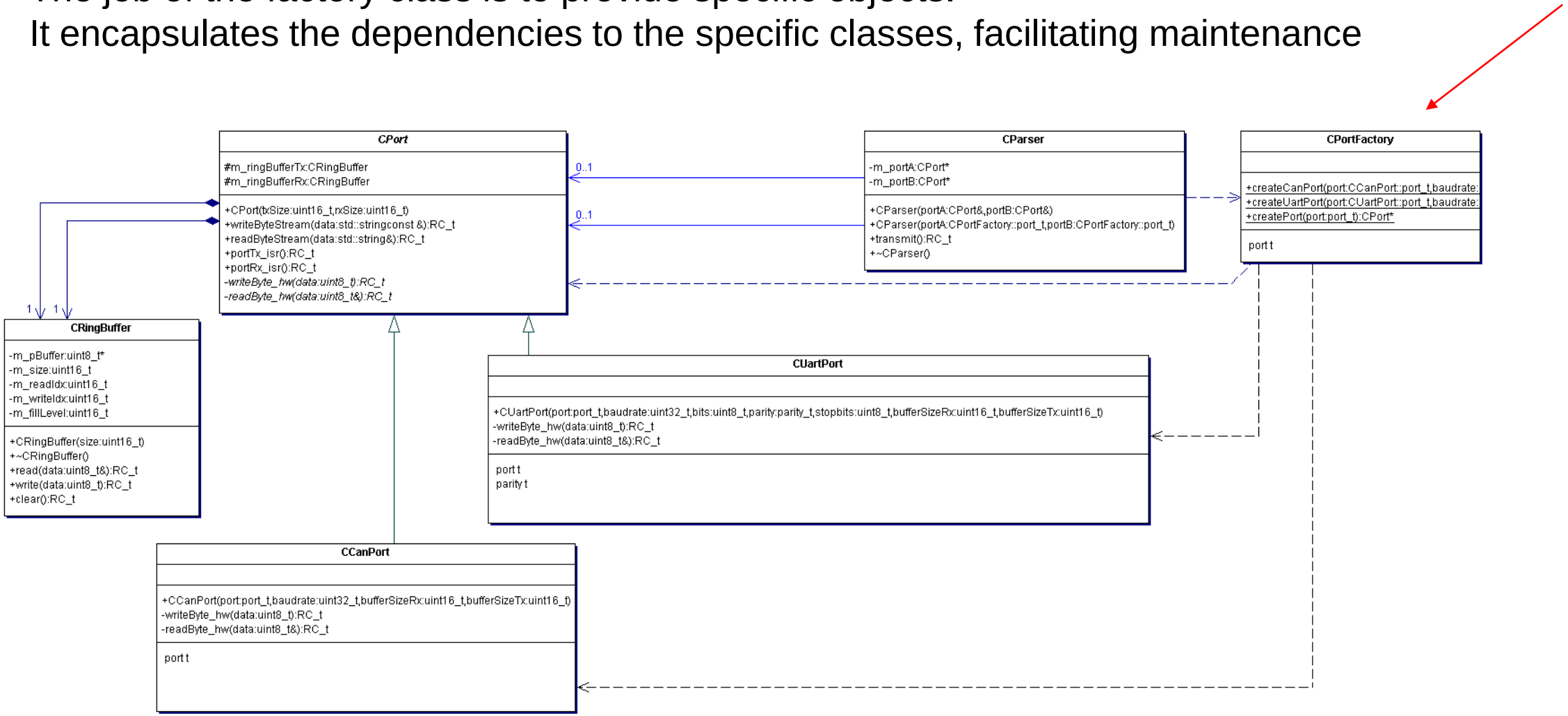
Dependency Issue

- But `main` (or any other class instantiating the hardware) has to include all physical devices – not so nice

```
#include "CPort.h"
#include "CParser.h"
#include "CUartPort.h"
#include "CCanPort.h"


// Main program
int main (void)
{
   //Let's create the parser
   CUartPort uart;
   CCanPort can;

   CParser parser(uart, can);
   parser.transmit();

   return 0;
}
```

Let’s introduce the Factory Pattern

- The job of the factory class is to provide specific objects.
- It encapsulates the dependencies to the specific classes, facilitating maintenance



Factory Pattern - Implementation

```
#include "CPort.h"
#include "CUartPort.h"
#include "CCanPort.h"


class CPortFactory {

public:

enum port_t{UART, CAN, NONE};

static CPort* createCanPort(
CCanPort::port_t port = CCanPort::NONE,
uint32_t baudrate = 1000000,
uint16_t bufferSizeRx = CAN_DEFAULTBUFFERSIZE,
uint16_t bufferSizeTx = CAN_DEFAULTBUFFERSIZE);

/**
 * Will create a port using the default configuration of the
    port (i.e. the default parameters set above)
 */
static CPort* createPort(port_t port);


};
```

```
CPort* CPortFactory::createCanPort(CCanPort::port_t port,
uint32_t baudrate, uint16_t bufferSizeRx, uint16_t
    bufferSizeTx)
{
return new CCanPort(port, baudrate,bufferSizeRx, bufferSizeTx);
}


CPort* CPortFactory::createPort(port_t port)
{
switch (port)
{
case UART: return createUartPort();
case CAN: return createCanPort();
default: /* Some error handling */ break;
}

return 0;
}
```

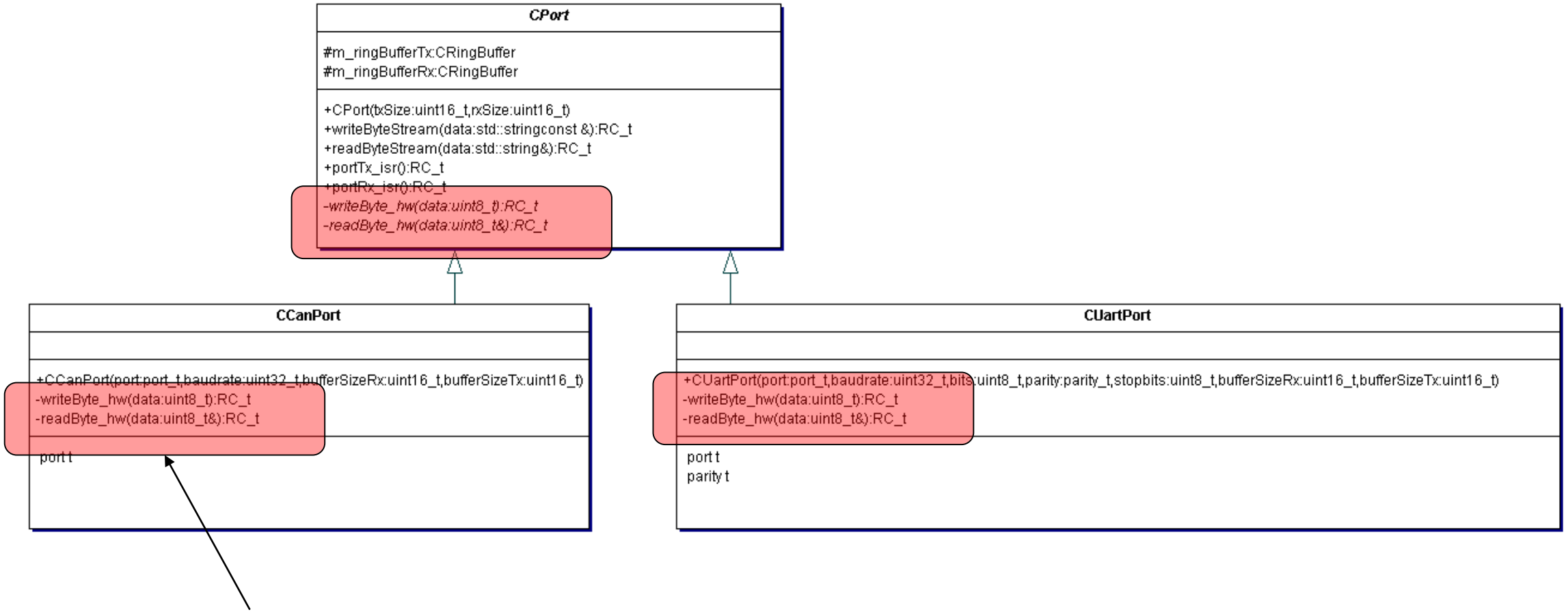
Interface Issue

What can we do?

I guess we have violated the Liskov Substitution Principle?

We have a problem: UART takes a single byte, CAN takes 8. Ethernet even more?

Liskov Violation



Here, we would like to have a 8byte API

How could we improve the API

Solution 1 – We accept the 1 byte API

- Would require 8 calls and some internal buffering for the CAN
- Difficult to synchronise

Solution 2 – pass a uint8_t pointer

- The driver must know, how much data can be read
- Very efficient but dangerous. What happens if we want to transmit a single uint32 over CAN?

Solution 3 – pass a stream object having an explicit length method

- Probably the best solution, as this is the common entity of a communication driver: transmit a specific number of bytes in one chunk
- Requires some extra effort
- But would also support a transport protocol (i.e. several packages transmitted one after the other)

A deeper look at method interfaces

- Minimise the scope, consider public versus private functions
- Avoid global data for passing information
- Provide self explaining names
- Provide abstract parameters, hiding implementation details
- Make it const
- Prefer call by value versus reference versus pointer
- Consider parameter passing conventions from the compiler
- Don’t forget error handling

Minimise the scope, consider public versus private functions

Goal

- Only functions / methods which should be called from the outside, should be made visible to the outside. This simplifies the usage of the module.
- This is also true for type definitions and data.

Recipe

- In C++, use private methods, type definitions and attributes
- In C, declare private types and functions in the C file (not Header). Use static data if needed.

Example

Statemachine.h

```
RC_t STATE_processEvent(EventType ev);
```

Statemachine.c

```
typedef enum {ISIDLE, ISPLAYING, ISRUNNING} STATE__state_t;

static STATE__state_t STATE_state = ISIDLE;

static void STATE__action1();
static bool STATE__guard1();
```

Avoid global data for passing information

Goal

- Using global data for passing information obfuscates the signal flow, as the user cannot easily find out, where the data is written to.
- Furthermore global data is the number one reason for potential races in multithreading environments.

Recipe

- Use Parameters for passing information to and from functions.
- Use references or pointers if performance is a genuine issue.

Example

```
void calculateFilter()
{
    switch (filterType){
    case MEAN: …
```

```
void calculateFilter(filter_t filterType)
{
    switch (filterType){
    case MEAN: …
```

Data Scope

Goal

- Reduce the scope of data as much as possible
- This will reduce of obfuscated coupling and potential races.

Recipe

- Avoid global data at all (this will probably not work)
- Reduce the scope
- Provide API's to access global data
- Provide semaphores as needed.

Example

```
static speed_t POOL__speed = 0;

RC_t getSpeed(speed_t& speed)
{
   GetRessource(speed_ressource);
   speed = POOL__speed;
   ReleaseRessource(speed_resource);
}
```

Provide self explaining names

Goal:

- There should be no confusion on how to call the function, even without reading the documentation
- Furthermore, good names help you to provide the correct content

Recipe

- Provide the name as mini sentence, consisting of a verb and a noun (recommended)
- Use CamelCase or _ to separate words
- Provide full parameter type AND and name in the API
- Introduce namespaces (Prefixes in C)

Examples

- `RC_t` **`CParser::transmit()`** versus **`CParser::transmitFromPortAtoPortB()`**
- `RC_t` **`CRingBuffer::write(uint8_t data)`**
- **`CCanPort::CCanPort(port_t port, uint32_t baudrate, uint16_t bufferSizeRx, uint16_t bufferSizeTx)`**
- **`CCanPort::CCanPort(port_t , uint32_t, uint16_t, uint16_t)`**

Provide self explaining names – Limitations and Solutions in C versus C++

C++ provide several layers of namespaces

- Global namespace (visible to everybody)
- Explicit namespace
- Class namespace
- Namespace::Class::Property

```
class Game
{
public:
    typedef enum {ISIDLE,
                  ISRUNNING} state_t
};

Game::state_t myState = Game::ISIDLE
```

In C, we only have the global namespace

- Therefore we have to introduce own namespace by naming convention
- Add a prefix to every (non local) property

Example

- `GAME_processEvent()`
- `typedef enum {GAME__ISIDLE, GAME__ISRUNNING} GAME__state_t;`
- `static GAME__state_t GAME_state = GAME__ISIDLE;`

Provide abstract parameters, hiding implementation details

Goal

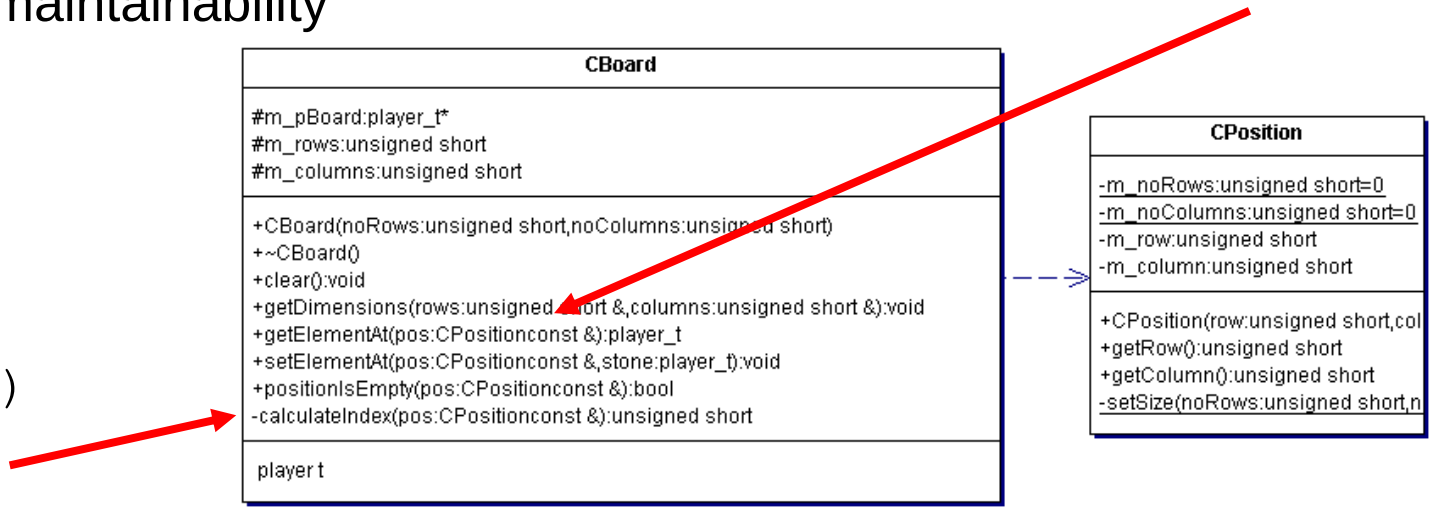
- The user shall be able to call the function without knowing (complex) internal details
- The module responsible shall be able to modify the internal behaviour without changing the API

Recipe

- Consider functionality and ignore implementation details when designing the API
- Use internal (private) methods to translate abstract parameters into implementation specific format
- Use abstract data types, supporting maintainability

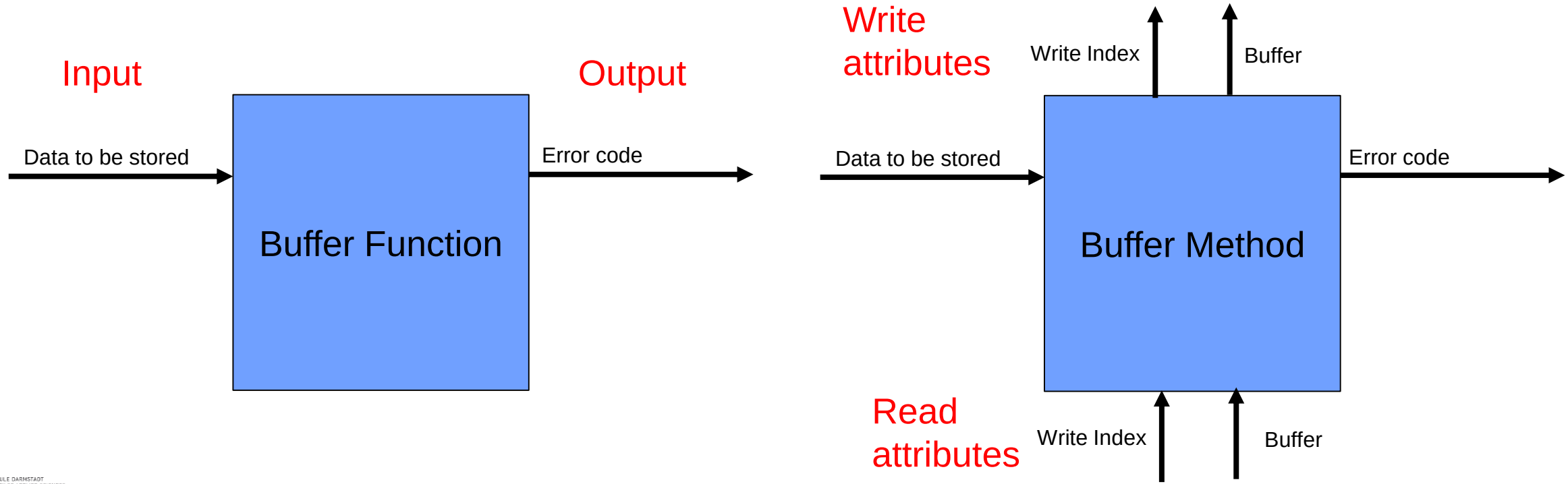
Examples

- `getElementAt(CPosition pos)`
- `getElementAt(uint16_t row, uint16_t col)`
- `getElementAt(uint16_t index)`



Interface Diagram

- Supports interface design in an abstract way
- Focussing on information flow, not implementation details
- Types may be added later (optional)
- Technical details like pointer, reference etc. may be added later (optional)



Make it const

Goal

- Accidental modification of data may cause system failures and is hard to debug.
- Therefore data, which is not intended to be modified, should be declared as const

Recipe

- Const comes in two different flavours
  - Data is linked into flash (making it really const)
  - The compiler is instructed not to modify data -creating compiler errors in case of violations. Problem: this const data can be casted into non-cast data or modified by pointers and therefore is not 100% safe.

Examples

- `const int calibrationCurve[] = {1,2,2,2,4,4,5,6,7,7,7}; //Very likely placed in flash`
- `void writeData(data_t const& data); //data reference should not be modified`

Make it const

```
//parameter data may not be changed inside the function
void foo(int const data)

//reference to parameter data may not be changed inside the function. Reference is used as read only
void foo(int const& data)

//Pointer address and dereferenced value may not be changed. Data is passed read only
void foo(int const * const data)

//Pointer address is constant but dereferenced value may be changed. Data is passed read / write
void foo(int * const data)

//Returned data may not be modified. E.g. for protecting a reference
const data_t& getSomeData()

//Method does not modify the object, i.e. method works with const and non-const objects
void class::foo() const
```

Const refers to the left hand type, unless this does not make sense. In this case, the right hand type is taken.

Make it const

Changing a non-const type to a const type during refactoring is frustrating

- May compilation errors
- Sequences of const’s are required in many files

Therefore

- Introduce const right away from the start of the project
- Make methods const if the do not modify the object

Prefer call by value versus reference versus pointer

Goal

- Call by reference and call by pointer are more performant, but may have side effects. They are not re-entrant and data accidentally may be overwritten.

Recipe

- For small data types, which fit into a register, a call by value should be used
- For larger datatypes a const& is the preferred approach (unless re-entrance is required)
- Pointers should only be used if references are not available or if the architectures requires it (e.g. to support NULL pointers in case of late associations)

Examples

- `void foo(uint16_t data) //Safe, re-entrant and no benefit when using references or pointers`
- `void foo(largeData_t const& data) //Fast, but special care needed in case of re-entrance`

Consider parameter passing conventions from the compiler

Goal

- Depending of the compiler and architecture, different conventions may be valid for passing parameters. This may have a significant impact on runtime.

Recipe

- Passing data by register is efficient.
- Passing large data structure by stack slows the execution down
- Multiple calls of copy and assignment operators slow the operation down

Examples

- ```
  void foo (class_t data) { m_data = data;}
      //Will use the copy constructor to copy the data on the stack
      //Then the local variable will be copied into the attribute
  ```
- ```
  void foo (uint32_t d1, uint32_t d2, uint32_t d3, uint32_t d4,
  uint32_t d5)
      //d1 until d4 will be passed by register on most 32bit machines
      //d5 will require an expensive stack operation
  ```

Don’t forget error handling

Goal

- Any function must ensure a safe operation under all conditions. In case a fault cannot be handled, it must be reported to the next “responsibility level”, the caller.

Recipe

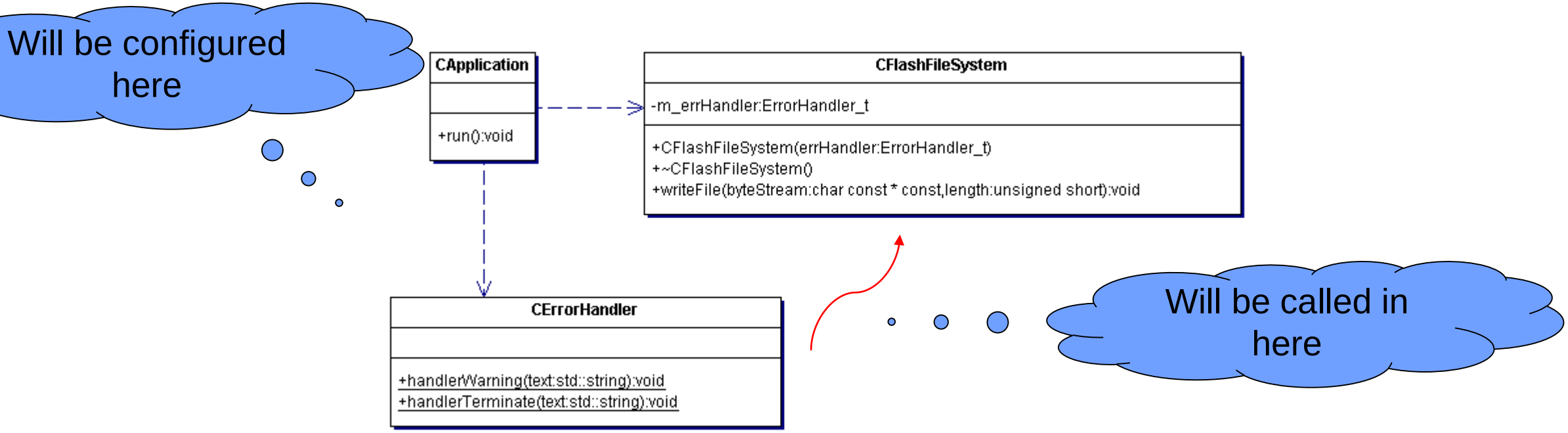
- Provide error handling for all functions, which might throw an error (very likely almost all)
- Define a common error handling concept as part of the architecture

Examples

- `RC_t foo(uint16_t value) { if (0 == value) return RC_BADPARAM; }`

Function Pointers and Functors

- Let’s assume, we are implementing a function, which is writing data on a flash disc.
- Depending on the application, different error handlers shall be called.
- We do not want to rely on the user to use the return value properly (as most programmers tend to ignore return values).
- Instead, the user shall provide a callback function, which will implement the error handler.



Function Pointer Declaration

```
class CFlashFileSystem
{
public:
/**
 * Function Pointer for the error handler
 */
typedef void (*ErrorHandler_t)(std::string errorMessage);

private:
ErrorHandler_t m_errHandler;

public:

CFlashFileSystem(ErrorHandler_t errHandler);

virtual ~CFlashFileSystem();

void writeFile(char const *const byteStream, unsigned short length);
};
```

Function Pointer Call

```
CFlashFileSystem::CFlashFileSystem(ErrorHandler_t errHandler)
{
m_errHandler = errHandler;
}

CFlashFileSystem::~CFlashFileSystem()
{
m_errHandler = 0;
}

void CFlashFileSystem::writeFile(const char *const byteStream,
unsigned short length)
{

//Todo: check for valid errHandler;

if (0 == byteStream) {
m_errHandler("Wrong param - byteStream points to 0");
}

if (0 == length) {
m_errHandler("Wrong param - nothing to write, length is 0");
}

}
```

Handler Implementation

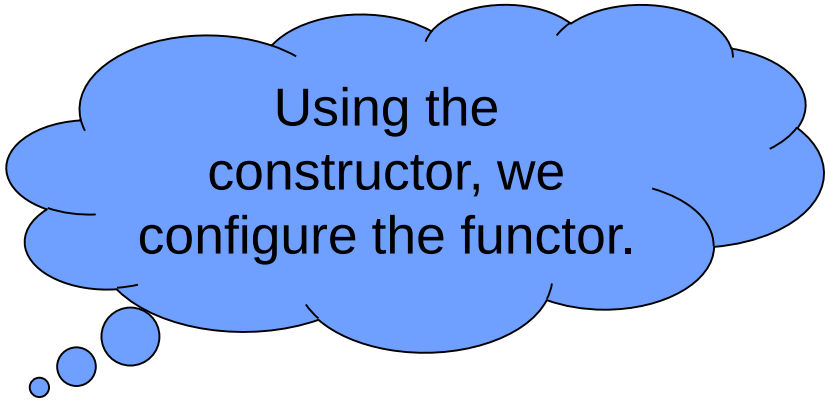
```
void CErrorHandler::handlerWarning(std::string text)
{
   cerr << "Error: " << text << endl;
}

void CErrorHandler::handlerTerminate(std::string text)
{
   cout << "Error: " << text << endl;
   terminate();
}
```

Functors

- In addition to function pointers, C++ provides so called **function objects**, aka functors.
- Basically, this is nothing but an overloaded () operator, which makes the class object behave like a function.
- The overloaded () can be considered to be the most flexible operator, as it imposes no limitations on the number and type of arguments.
- Also the return type is user defined.
- As compared to ordinary function pointers, a function object may contain additional attributes, reflecting an internal state of the function.
- Furthermore these attributes may be set when calling the constructor.

Error handler class using functors

```
class CFunctorErrorHandler
{
private:
   bool m_terminateOnError;
public:
   CFunctorErrorHandler(bool terminateOnError = true);

   void operator()(std::string msg);
};

CFunctorErrorHandler::CFunctorErrorHandler(bool terminateOnError)
{
     m_terminateOnError = terminateOnError;
}

void CFunctorErrorHandler::operator ()(std::string msg)
{
cerr << "ERROR: " << msg << endl;

if (true == m_terminateOnError) {
terminate();
}

}
```



Calling it in the application

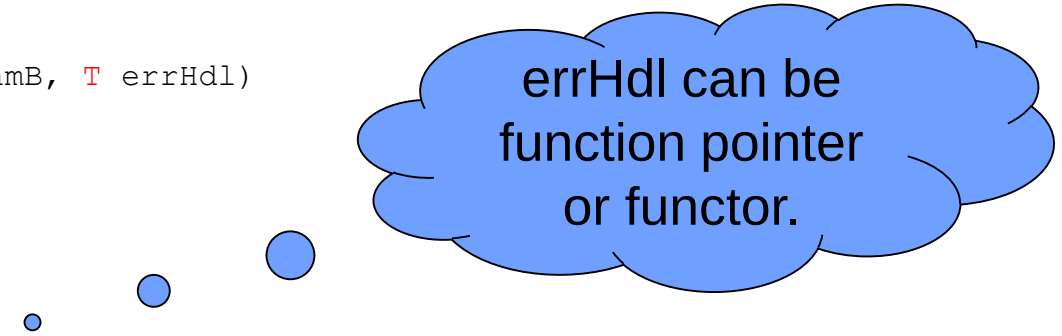
```
void CFlashSystemFunctor::writeFile(const char *const byteStream,
unsigned short length)
{
//Todo: check for valid errHandler;

if (0 == byteStream) {
        m_errHandler("Wrong param - byteStream points to 0");
}

if (0 == length) {
        m_errHandler("Wrong param - nothing to write, length is 0");
}
}
```

Advantage of the functor – the termination logic is encapsulated in the class, reducing the user flexibility a bit. Which sometimes is not too bad!

Mixing Function Pointers and Functors

- Although functors share the signature of ordinary function pointers, they cannot be exchanged. For this, we will need the argument deduction of templates.

```
template <class T>
void functionTakingAnyErrorHandler(int paramA, int paramB, T errHdl)
{
if (0 == paramA)
{
   errHdl("A was 0");
}

if (0 == paramB)
{
   errHdl("B was 0");
}
}

//Using Fuction Pointers
functionTakingAnyErrorHandler(0,0,CErrorHandler::handlerWarning);
functionTakingAnyErrorHandler(0,0,CErrorHandler::handlerTerminate);

//Using Functors
functionTakingAnyErrorHandler(0,0, CFunctorErrorHandler(false) );
functionTakingAnyErrorHandler(0,0, CFunctorErrorHandler(true) );
```



Functors and Polymorphism?

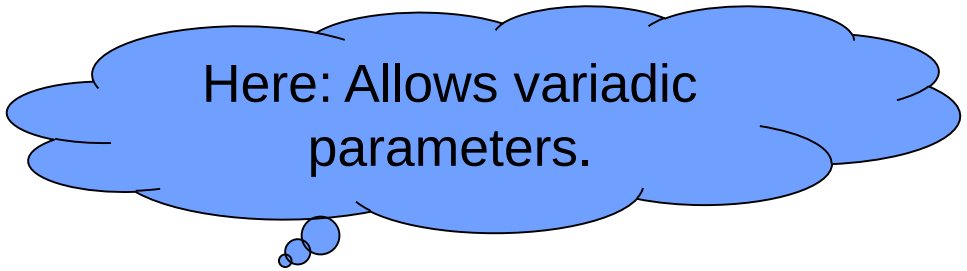
- We might create a error handling base class, defining the functor API
- And then allow the user to implement specialised error handling classes
- The application code will call the base class Functor
- Which will call specialised error handlers if required.

How about Lamda‘s

- C++11 introduces lambdas allowing you to write an inline, anonymous functor to replace the explicit class declaration.
- Good for e.g. filters and similar “small logic”

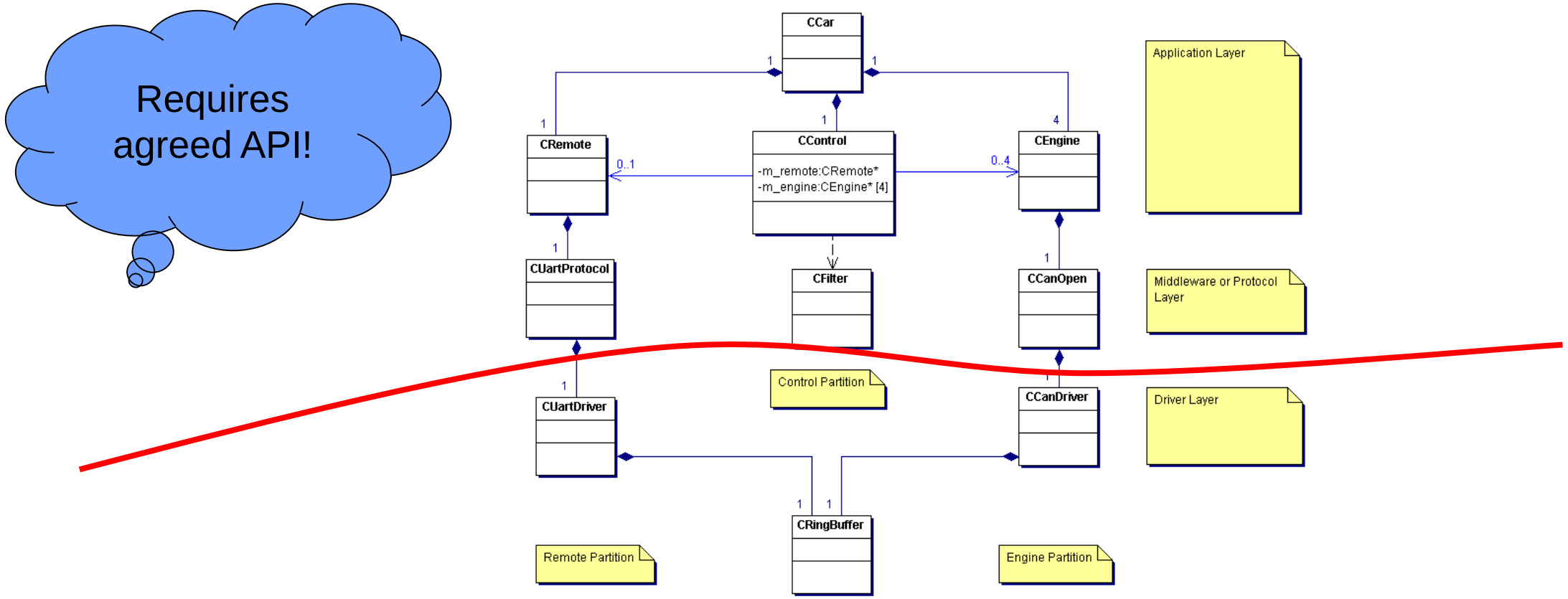
- Example: Deadline Monitoring of runnables having different API’s

```
int MonitorStart(int p_ret, RC_t (*p_run)())
{
         PxExpectAbort(RunWrapper, p_ret, p_run);
        return 0;
}
void RunWrapper(int* p_ret, R (*p_run)())
{
*p_ret = p_run();
}

int retVal;
MonitorStart( &retVal, []() -> int {return RunDiverse(2, 3.0f, 'c');});
```



Let’s come back to earth….

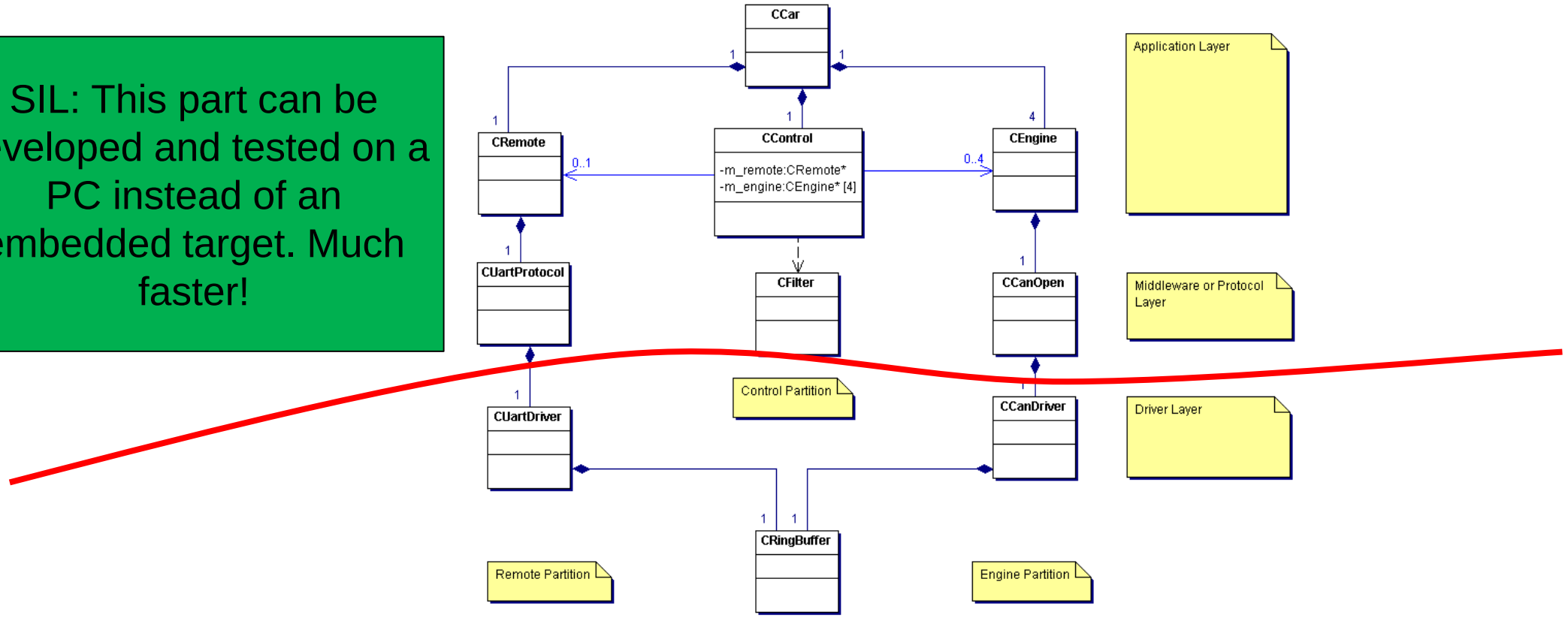
Stubbing



Very likely, the drivers will not be present upon project start. Instead, we can agree on the API and use stubs instead.

Stubbing

SIL: This part can be developed and tested on a PC instead of an embedded target. Much faster!



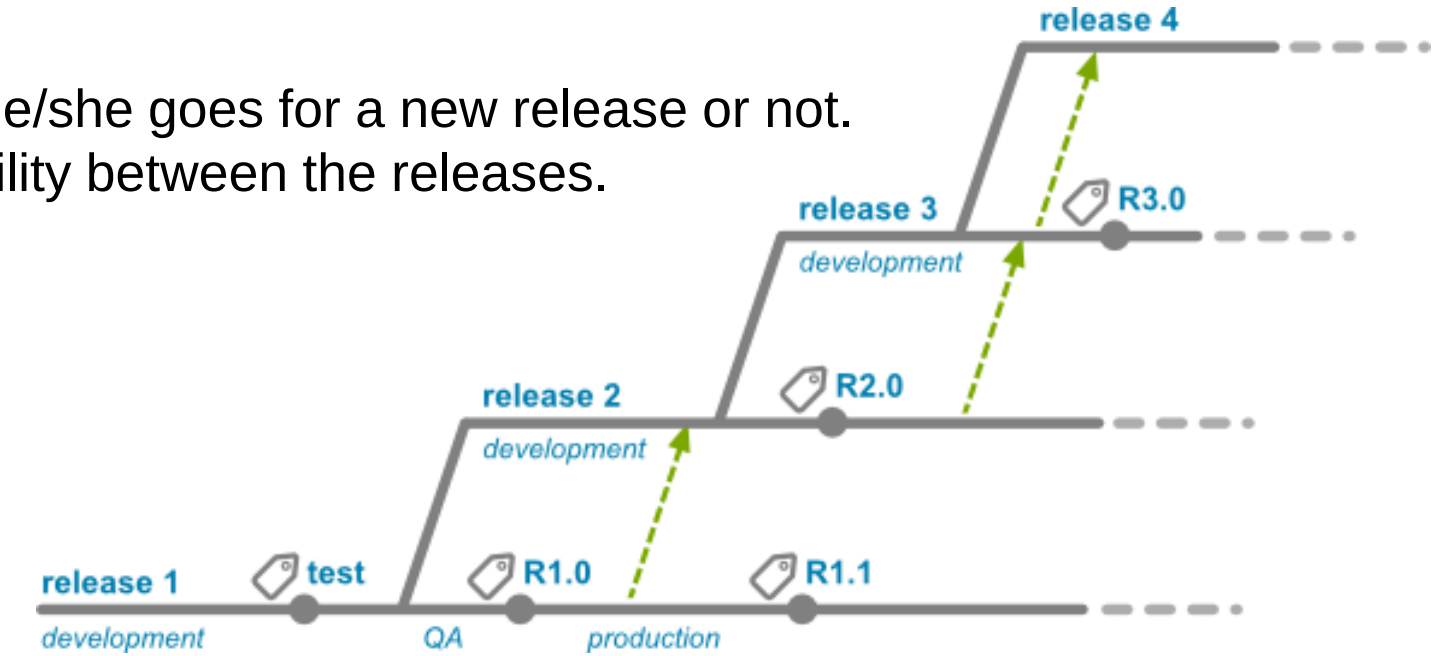
Provide simulated drivers on PC side. This also allows to test functionality, but probably not the timing aspects, memory, full error handling…

Release process

Remember….

- The classes / sources which form the component must go through a release process
- The class releases must form a working component release. The classes have to be compatible!
- This implies a strong cohesion inside the component. It has to make sense, that these classes are released together.

- Finally, the developer will decide if he/she goes for a new release or not.
- This implies also a certain compatibility between the releases.



The Common Closure Principle

- Only classes, which change for the same reason and at the same time should be placed into a component.
- Classes, that change for different reasons or at different times should be placed in separate components.

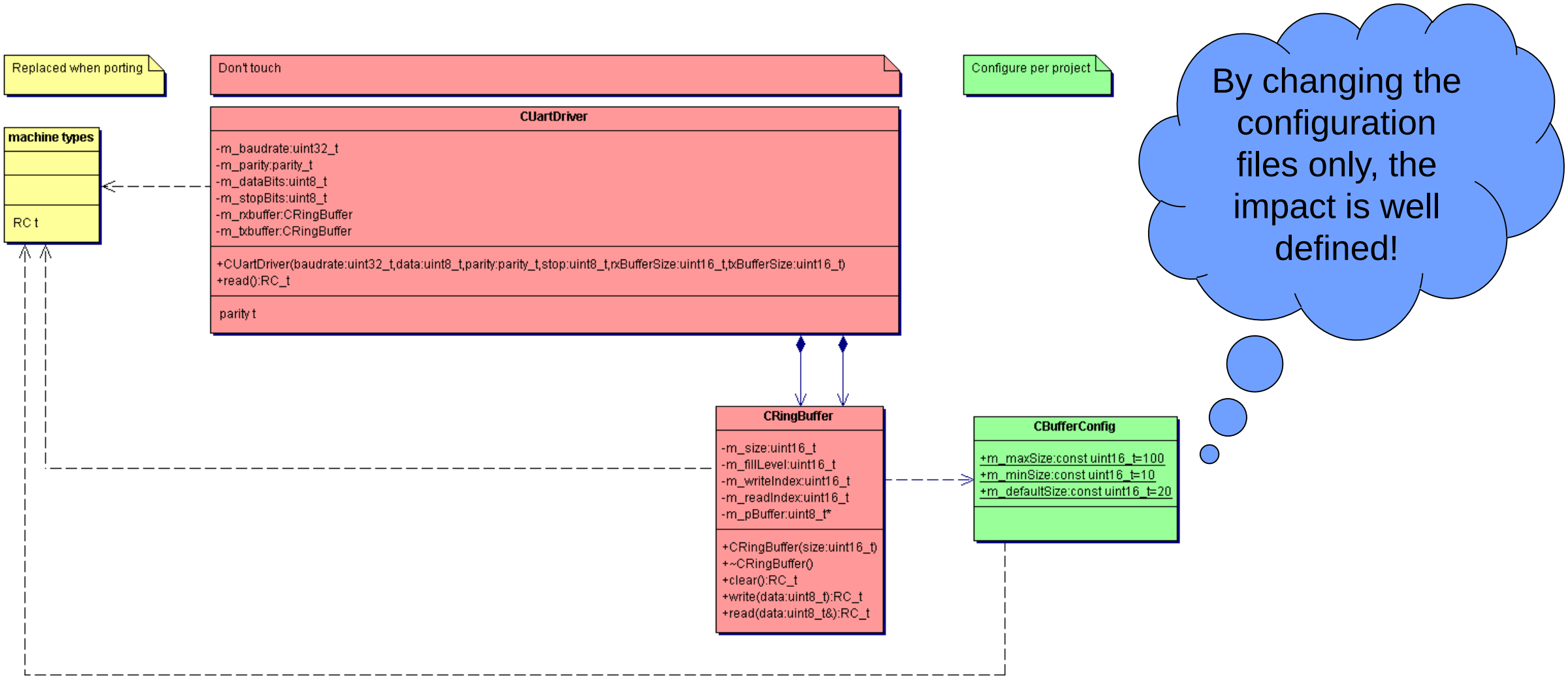
Same is true for class design. A good example is the separated configuration of a class!

Configurations and Variants

Until now, the main concept for generalisation and specialisation was inheritance. However, there are way more options:

- Configuration by parameters (e.g. constructor) ➔ valid for a specific object
- Separation of specific configuration parts (enums, #defines, const tables) in own files ➔ valid for the complete class (i.e. all objects)
- Classes, sharing similar behaviour but different data structure ➔ Templates

Configuration Files



Project Level versus User Level Configuration

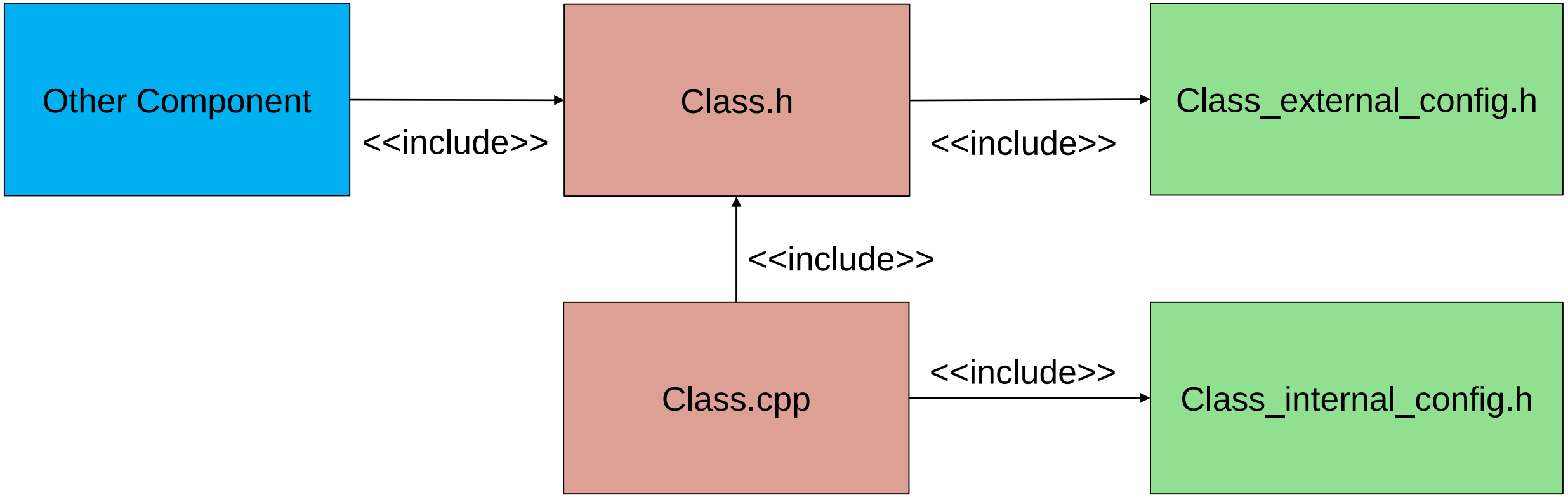
This parameter is configured by the individual user (per object)

```
CRingBuffer::CRingBuffer(uint16_t size)
{
if (size >= CBufferConfig::m_minSize && size < CBufferConfig::m_maxSize)
{
m_size = size;
}
else
{
m_size = CBufferConfig::m_defaultSize;
}

m_fillLevel = 0;
m_readIndex = 0;
m_writeIndex = 0;

m_pBuffer = new uint8_t[m_size];
}
```

This is a constraint on class / project level. May be different for different projects.

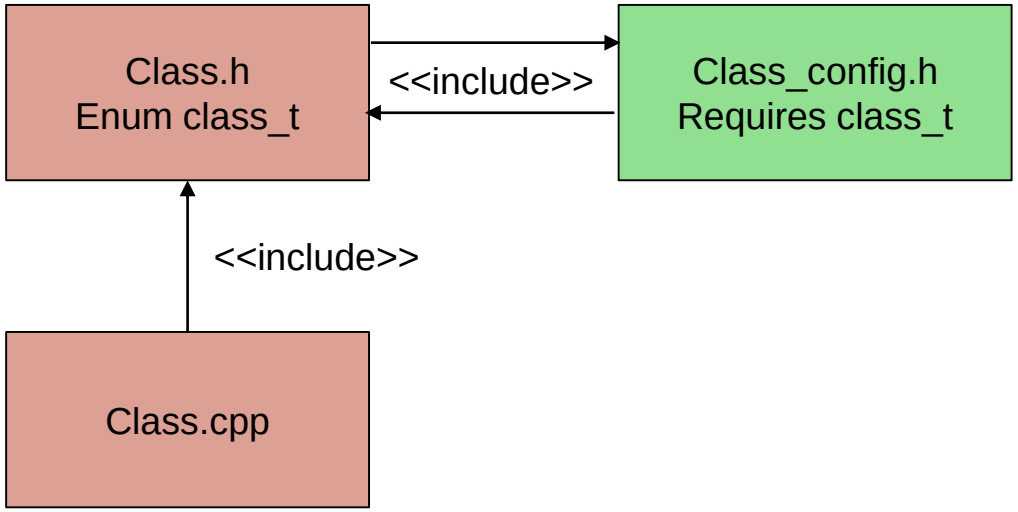
Configuration Files – Internal versus External Configuration

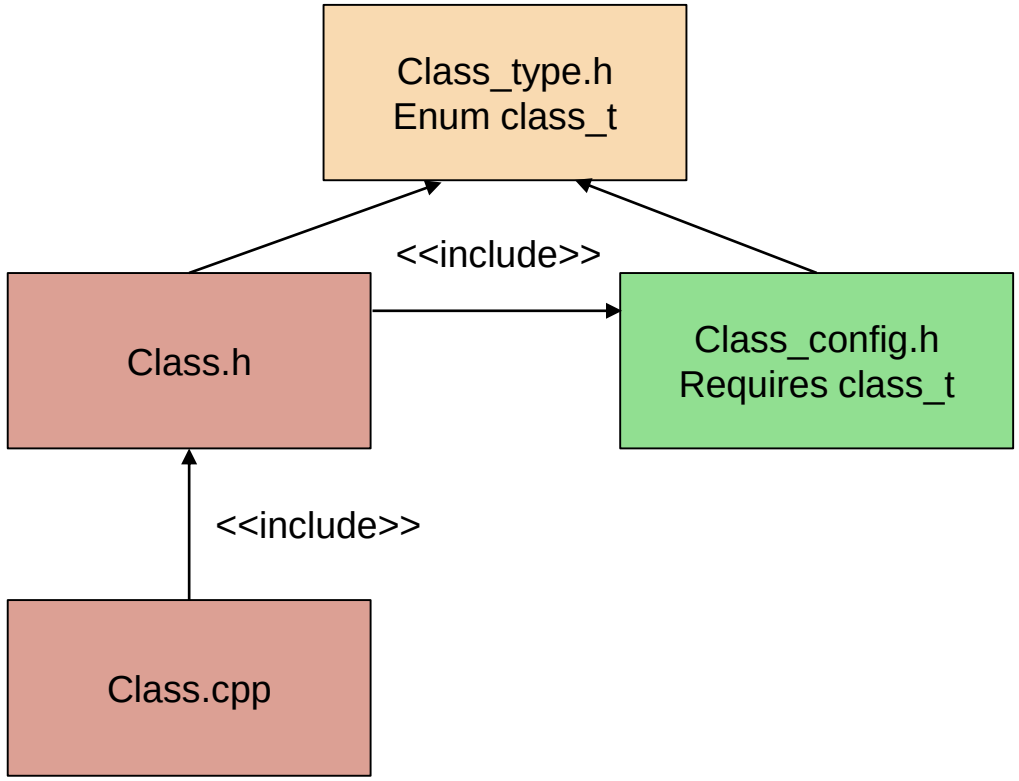
Some configurations should be visible to the complete project, other configurations should be private for the component.



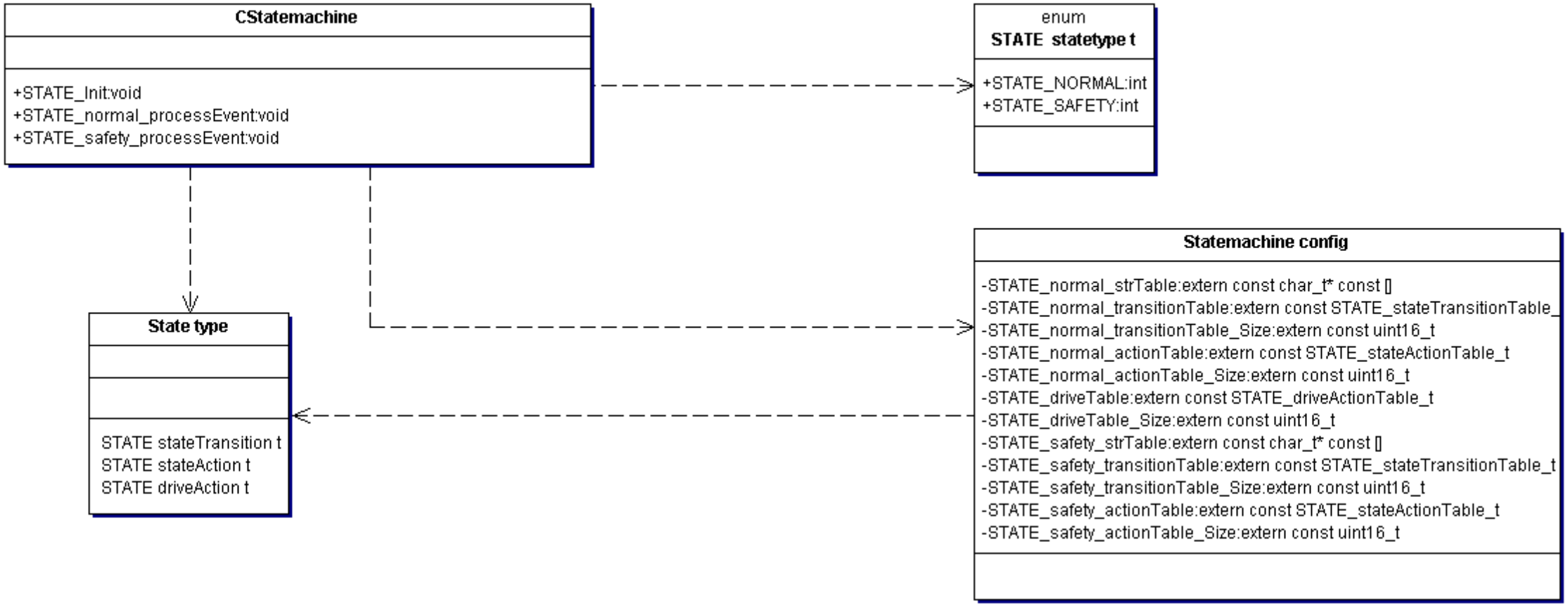
Separation of Type Headers

- Very often, a foreign component or a configuration class requires some type definitions of another components, e.g. enum values. To avoid circular dependencies, these can be placed in an extra file.
- Also speeds up compilation, as only the type file is included!





Configuration Files Containing Code



Configuration Files Containing Code – Example Statemachine Logic

```
const STATE_stateTransitionTable_t STATE_normal_transitionTable =
{

//Event         //fromState                 //Guard                             //Action                                                    //toState
//=============================================================================================================================================================

        {/* evNMTBooted, IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0 /* Action will be called in Communication Task */ , (uint32_t) IS_DEVICES_BOOTING },
        {/* evDeviceBoot,IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evPresent,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evConnect,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evFeatReq,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evFeatAct,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evPreRunChk, IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evStop,      IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evDisconn,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },
        {/* evEmBrake,   IS_NMT_BOOTING    */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_NMT_BOOTING },


        {/* evNMTBooted, IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evDeviceBoot,IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0 /* Action will be called in Communication Task */, (uint32_t) IS_DISCONNECTED },
        {/* evPresent,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evConnect,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evFeatReq,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evFeatAct,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evPreRunChk, IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evStop,      IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evDisconn,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },
        {/* evEmBrake,   IS_DEVICES_BOOTING */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DEVICES_BOOTING },


        {/* evNMTBooted, IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evDeviceBoot,IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evPresent,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) STATE__normal_sendPresentAck,  (uint32_t) IS_PRESENT },
        {/* evConnect,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evFeatReq,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evFeatAct,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evPreRunChk, IS_DISCONEECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evStop,      IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evDisconn,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },
        {/* evEmBrake,   IS_DISCONNECTED */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_DISCONNECTED },

        {/* evNMTBooted, IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evDeviceBoot,IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evPresent,   IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evConnect,   IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) STATE__normal_sendConnectAck,  (uint32_t) IS_CONNECTED },
        {/* evFeatReq,   IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evFeatAct,   IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evPreRunChk, IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evStop,      IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) 0, (uint32_t) IS_PRESENT },
        {/* evDisconn,   IS_PRESENT      */(STATE_GuardPtr_t) 0, (STATE_ActionPtr_t) STATE__normal_sendDisconnectAck, (uint32_t) IS_DISCONNECTED },
```

Another example

Let‘s consider a generic PID controller for an engine…

Which parts must be configurable?

- Constant Data
- Changing Data
- Data Types
- Behaviour
- API restrictions (e.g. from a generated driver?)

Wrapup

- Interface design on component, class as function level
- Golden rules for API design
- Function parameters or functors adding flexibility
- Separating configuration parts